

# Lab 1 : 词法与语法分析

---

## 《编译原理》

---

姓名 / 学号 : 林武文 / 3230300081

专业 : 计算机科学与技术

日期 : 18/3/2026

---

## Lab Introduction

We will input a `sysY` source file and do some lexical analysis + syntax analysis + build AST

**a.k.a** : checks whether the source program is lexically and grammatically valid, and if valid it builds the corresponding AST

## Lab Approach / 试验思路

---

### 1. `main.cpp`

This is the driver of our compiler which connects user command with lexer and parser, calls `yyparse()` which triggers the next few steps (parsing), any parsing errors would result in error code `1`

### 2. `lexer.l`

We start off with **lexical analysis** within this file, which does :

- scans the source file character by character
- then we group them and assign each a token

e.g. :

- `"==" { return EQ; }` : this is a operator tokens
  - This maps a string pattern to a token `EQ` , then the parser can recognize the operator

We also define the regex for simplifying the rule, e.g. :

- `digit [0-9]` : this resemble a digit and will be used to simplify :
  - `identifier {letter}({letter}|{digit})*`

### 3. parser.y

We perform the `syntax analysis` here, which does :

- receives the tokens from `lexer.l`
- and checks whether they match the grammar rules, and finally
- constructs `AST` nodes

e.g. : Addition rules

```
AddExp : AddExp "+" MulExp {  
    $$ = new BinaryExp(BinaryOp::Add, NodePtr($1), NodePtr($3));}
```

- for matching `a + b`, we create a `BinaryExp` AST node which connects both `a` and `b` together

Not forgetting the `precedence` between operators by assigning them sequentially (based on precedence)

### 4. tree.hpp + AST

This where we define the abstract syntax tree nodes

**Each node stores** information based on the grammar structure, printable string for debugging

e.g. : `CompUnit` is the program root, `FuncDef` is the function, `VarDecl` is the variable declaration, etc.

- Each node may call `get_children()` defines the tree structure

e.g. : After compiling `add.sy` file (output is the `AST` )

```
CompUnit (line 8)  
└─ FuncDef <return_btype: int, name: main> (line 8)  
    └─ Block (line 3)  
        └─ VarDecl <btype: int> (line 3)  
            │   └─ VarDef <ident: a> (line 3)  
            │       └─ VarDef <ident: b> (line 3)  
<SNIPPED>
```

- We can see the `main` as `FuncDef` type `int`, `Block` node represents the function body (enclosed by braces)
  - inside it contains the `VarDecl` which is the variable declaration, and some Assigning + Expression statement, ended with the Return statement ( `0` )

## Difficulties and Highlights / 难点与亮点

---

### (-) Declarations and Function Definitions

- e.g. : `int main` earlier can either be a variable or a function definition
- So in many test cases when it reaches `(`, it return 1 as syntax error

→ I wrote the function definition using `int` and `void` so it became easier to distinguish function definitions and variable definitions

### (+) I've improved the Lab1 from template

- By modifying : `common.hpp` (operator representation), `lexer.l` (lexical analysis), `parser.y` (grammar analysis), `tree.hpp` (AST structure), Now I've built complete a complete AST.

(-) Although I'm not sure with the array parsing since that is not the primary objectives on Lab 1, so I'll leave it be for the next lab

## Reflection / 心得体会

---

Throughout this lab I feel very confused for a moment, because the syntax is new and quite abstract. But it took sometimes to finally understand what I'm doing, this lab made me understand why lexical analysis and parsing roles are important before constructing the AST, and why the sequence of steps taken is also important, e.g. if token is wrong in the lexer, parser may still fail even when the grammar is correct. I also understand what it means to build it incrementally from simple functions and expressions to control flow and arrays is much more manageable.

## Additional Content / 附加内容

---

借鉴声明 : I didn't have any reference while doing this lab

## AI 工具使用记录 : I only use ChatGPT (GenAI) for assist during the lab

For each code, the first step is to ask the basic syntax of the file e.g. : `lexer.l`

Show me the syntax from a lexer.l file

A `lexer.l` file (used with **Lex/Flex**) follows a very standard structure. It is divided into **three sections**, separated by `%%`.

### Basic Structure of `lexer.l`

```
/* C
/* C declarations */
#include <stdio.h>
#include <stdlib.h>
*/

/* Definitions (optional) */
DIGIT  [0-9]
LETTER [a-zA-Z]

%%

/* Rules section */
{DIGIT}+ { printf("Number: %s\n", yytext); }
{LETTER}+ { printf("Word: %s\n", yytext); }
"+" { printf("Plus operator\n"); }
"-" { printf("Minus operator\n"); }
[ \t\n] { /* ignore whitespace */ }
. { printf("Unknown: %s\n", yytext); }

%%
```

If I do have some confusion I just directly asked the syntax or the logic behind it :

```
{identifier} { yylval.str_val = strdup(yytext); return IDENT; }
{linecomment} {}
{comment} {}
{blank} {}
. { std::string msg = "unknown character '" +
std::string(yytext) + "'"; yyerror(msg.c_str()); exit(1); }
```

can you explain the format here ?

Good question — this is the **core of how a lexer actually works**.

Let's break the *format* first, then each line.

### General Format of a Lexer Rule

In Flex, every rule follows this structure:

```
lex
PATTERN { ACTION }
```

- PATTERN** → a regex (what to match)
- ACTION** → C/C++ code (what to do when matched)

So your block:

```
lex
```

After every finished code, I'm making sure whether I've fulfilled the requirements for the current file, or whether I have a typo or syntax error :

```

{identifier} { yyval.str_val = strdup(yytext); return IDENT;
}
{linecomment} {}
{comment} {}
{blank} {}
. { std::string msg = "unknown character '" +
std::string(yytext) + "'"; yyerror(msg.c_str()); exit(1); }

%%

```

does my code here already satisfy the syntax of a lexer ?  
am I missing something ?

Yes — this **already has the correct overall Flex/Lex syntax**.

Your file has the standard lexer structure:

1. **options**
2. **C/C++ declarations inside** `%{ ... %}`
3. **macro definitions**
4. `%%`
5. **rules**
6. `%%`
7. **user code section** at the end

So in terms of **syntax shape**, it is a valid `lexer.l` layout.

But there are a few things that are **missing**, **⬇ plicated / potentially problematic**.

During the `parser.y` I knew I had problems with the definitions so I just straight up asked ChatGPT, but turns out only a small tweaks needed to fix the issues (resolved by ChatGPT as well) :

```
//test add
int main(){
  int a, b;
  a = 10;
  b = -1;
  write(a+b);
  return 0;
}
```

————→ \$./compiler sp26-tests/tests/lab1/add.sy  
Error at line 2: syntax error, unexpected (, expecting ; or ", "  
Parse failed with status 1



Good — this error is **very clear and very fixable** 🍌

## ✖ Your error

↗ Bash



Error at line 2: syntax error, unexpected (, expecting ; or ", "

Code:

↗ C



int main(){



```
| LOrExp "|" LAndExp { $$ = new
BinaryExp(BinaryOp::LOr, NodePtr($1), NodePtr($3)); }
;

%%

void yyerror(const char *s) {
  std::cerr << "Error at line " << yylineno << ": " << s <<
  std::endl;
}
```

Thought for 23s >

Yes — you **are missing something important**, and it is **not** the `CompUnit` order.

The real problem is:

**The parser is getting a conflict between `BType` and `FuncType`**

At the top level, both of these can start with `"int"`:

↗ bison



```
BType    : "int"
FuncType  : "int" | "void"
```

So when parsing:

↗ C

